

# Federated Supply Chain Milk Phase 2

Privacy-Preserving Demand Forecasting & Optimization

*Federated Learning | Differential Privacy | Local AI Insights*

|               |                                                |
|---------------|------------------------------------------------|
| Project Type: | Federated Learning Supply Chain Demo           |
| Domain:       | Milk Distribution & Demand Forecasting         |
| Architecture: | Control Tower + Client Node Network            |
| AI Engine:    | Local GGUF Model via llama.cpp CPU             |
| Forecasting:  | LSTM Neural Network (PyTorch)                  |
| Privacy:      | Differential Privacy + Encrypted Communication |
| Date:         | April 2026                                     |

. Table of Contents

|     |                               |    |
|-----|-------------------------------|----|
| 1.  | Executive Summary             | 3  |
| 2.  | Project Overview              | 3  |
| 3.  | System Architecture           | 4  |
| 4.  | Core Components               | 5  |
| 5.  | Federated Learning Pipeline   | 7  |
| 6.  | LSTM Forecasting Model        | 8  |
| 7.  | Optimization Engine           | 9  |
| 8.  | Privacy & Security Model      | 10 |
| 9.  | Secure Communication Protocol | 11 |
| 10. | Dashboard & UI Design         | 12 |
| 11. | Control Tower Application     | 13 |
| 12. | Client Node Application       | 14 |
| 13. | Dataset Specifications        | 15 |
| 14. | Local AI / LLM Integration    | 16 |
| 15. | Environment & Configuration   | 17 |
| 16. | Installation & Deployment     | 18 |
| 17. | Demo Workflow                 | 19 |
| 18. | Troubleshooting Guide         | 20 |
| 19. | Technical Specifications      | 20 |
| 20. | Future Roadmap                | 21 |

## 1. Executive Summary

This document presents the Federated Supply Chain Milk Phase 2 project - a comprehensive demonstration of privacy-preserving federated learning applied to supply chain demand forecasting and optimization in the milk distribution domain.

The system implements a full-stack architecture comprising a centralized Control Tower and distributed Client Nodes, connected through encrypted communication channels. At its core, the platform leverages Federated Averaging (FedAvg) with differential privacy to collaboratively train an LSTM demand forecasting model across multiple client participants without exposing individual raw data.

The project uniquely combines multiple advanced capabilities: federated machine learning, differential privacy noise injection, secure encrypted and HMAC-signed communication, local AI-powered strategic insights via a GGUF language model, financial and emissions optimization, and a human-in-the-loop override mechanism - all orchestrated through a modern Streamlit-based dashboard interface.

> Key Innovation: Unlike traditional supply chain analytics that require centralized data, this system enables multiple milk distribution clients to benefit from collective intelligence while preserving the privacy of each participant's proprietary business data.

### Key Highlights

- Federated learning with FedAvg aggregation across configurable client participants
- Differential Privacy (Laplacian noise) applied to model weight updates with tunable epsilon
- LSTM-based demand forecasting with 5-point sliding window sequences
- Supply chain optimization incorporating safety stock, disruption risk, and emissions
- Encrypted Fernet + HMAC-SHA256 signed communication between tower and nodes
- Local GGUF model (SupplyChain-Qwen) for strategic AI explanations via llama.cpp CPU
- Privacy-safe market benchmarking with minimum cohort enforcement
- Human-in-the-loop approval and override with persistent decision logging
- Real-time node health monitoring with online/offline status tracking
- Secure raw dataset upload from client nodes to the control tower

## 2. Project Overview

The Federated Supply Chain Milk Phase 2 project simulates a milk supply chain network where multiple client nodes contribute data to a federated learning process. Each client represents a milk distribution brand or regional entity that maintains its own identity and data sovereignty.

The system ensures that instead of sharing raw business data across participants, the control tower trains a shared forecasting model using federated techniques, and only privacy-safe market summaries are distributed back to individual clients for benchmarking.

### Core Value Proposition

- Supply chain analytics without centralized data exposure

- Federated learning education and demonstration platform
- Privacy-preserving benchmarking using aggregated and DP-sanitized values
- Local AI-assisted decision support without cloud dependency
- End-to-end secure communication for all tower-node interactions

### Target Use Cases

---

- Multi-brand milk distribution demand forecasting
- Privacy-compliant supply chain collaboration between competing entities
- Federated learning research and educational demonstrations
- Sustainable supply chain optimization with emission tracking
- AI-augmented supply chain decision support with human oversight

### 3. System Architecture

The system follows a hub-and-spoke architecture with the Control Tower as the central coordinator and multiple Client Nodes operating as distributed participants. All communication flows through encrypted and signed channels.

#### Architecture Overview

The architecture consists of three primary layers:

- i

Presentation Layer: Streamlit-based web dashboards for both Control Tower and Client Nodes, featuring a custom dark-theme UI with Inter font, card-based layouts, and real-time metrics.
- i

Business Logic Layer: Federated learning simulation (FedSim), LSTM forecasting, optimization engine, differential privacy module, market benchmarking, and local LLM integration.
- i

Communication Layer: Encrypted HTTP endpoints for health checks (/health), bundle sync (/sync), and raw file upload (/upload), using Fernet encryption and HMAC-SHA256 signatures.

#### Data Flow Architecture

The complete end-to-end data flow follows this sequence:

- 1. Client nodes upload raw CSV/XLS/XLSX files to Control Tower via encrypted /upload endpoint
- 2. Control Tower saves uploads in DATASETS/ with client-specific naming convention
- 3. Control Tower runs federated learning simulation across all client datasets
- 4. Each client dataset is treated as one federated participant in training rounds
- 5. LSTM model is trained round-by-round with differential privacy noise on weight updates
- 6. Tower generates privacy-safe market benchmarks using aggregation and DP sanitization
- 7. Tower securely sends each client its own benchmark bundle via encrypted /sync endpoint
- 8. Client app receives bundle and displays private metrics vs market comparison
- 9. Local AI model generates strategic insights from client + market aggregate data only

## 4. Core Components

### Project File Structure

| File / Directory   | Purpose                                  | Lines / Size          |
|--------------------|------------------------------------------|-----------------------|
| app.py             | Control Tower Streamlit Application      | 930 lines             |
| client_app.py      | Client Node Streamlit Application        | 432 lines             |
| main.py            | Core FL, Forecasting, Optimization Logic | 723 lines             |
| privacy_network.py | Secure Communication Layer               | 555 lines             |
| dashboard_data.py  | Market Benchmarking & Bundles            | 361 lines             |
| requirements.txt   | Python Dependencies                      | 11 packages           |
| DATASETS/          | Raw Client Dataset Files                 | 4 files               |
| models/            | GGUF Local Language Model                | SupplyChain-Qwen.gguf |
| Llama-cpp-cpu/     | Llama.cpp CPU Runtime                    | Bundled binary        |
| sc50_logs/         | Logs, Registry, Decision Logs            | 3 files               |

### Module Dependency Map

The codebase follows a clean dependency hierarchy:

- app.py imports from: main.py, dashboard\_data.py, privacy\_network.py
- client\_app.py imports from: main.py, dashboard\_data.py, privacy\_network.py
- dashboard\_data.py imports from: main.py (SCConfig, optimize, DifferentialPrivacy)
- privacy\_network.py is self-contained (uses cryptography.fernet for encryption)
- main.py is the foundational module (SCConfig, LSTMModel, FedSim, LLM loading)

### Configuration System (SCConfig)

All project-wide configuration is centralized in the SCConfig dataclass located in main.py. This includes model paths, financial parameters, privacy budgets, and directory paths.

|               |                                           |
|---------------|-------------------------------------------|
| MODEL_NAME    | SupplyChain-Qwen (GGUF via llama.cpp CPU) |
| PRODUCT_NAME  | Milk (Fixed Product)                      |
| NUM_CLIENTS   | 3 (Default, configurable 1-10)            |
| NUM_ROUNDS    | 2 (Default, configurable 1-10)            |
| CARBON_CAP    | 500.0 per batch                           |
| DP_EPSILON    | 5.0 (Lower = More Privacy/Noise)          |
| SELLING_PRICE | \$4.00 per unit                           |
| COST_PRICE    | \$1.50 per unit                           |
| WASTE_COST    | \$0.50 per unit (spoilage)                |
| DATASET_DIR   | DATASETS/                                 |
| LOG_DIR       | sc50_logs/                                |

## 5. Federated Learning Pipeline

The federated learning pipeline is implemented in the FedSim class within main.py. It uses the Federated Averaging (FedAvg) algorithm to collaboratively train an LSTM demand forecasting model across multiple client participants.

### FedAvg Algorithm Implementation

For each federated round, the following steps are executed:

- 1. The Control Tower broadcasts the current global model weights to all clients
- 2. Each client trains a local copy of the LSTM model on its own demand data
- 3. Local training runs for 5 epochs per round with Adam optimizer (lr=0.01)
- 4. Differential privacy noise (Laplacian) is injected into the client's weight updates
- 5. The tower collects all noisy client weights and computes the federated average
- 6. The global model is updated with the averaged weights
- 7. Validation metrics (MAE, RMSE, Loss) are computed across all client datasets

### Differential Privacy Integration

Privacy is enforced at the weight update level. After each client completes local training, Laplacian noise is added to every weight tensor before submission to the aggregator. The noise scale is controlled by the epsilon parameter:

```
# Noise injection per weight tensor
noise = torch.tensor(
    np.random.laplace(0, 0.01 / epsilon, weight.shape)
).float()
weight += noise
```

The DifferentialPrivacy class also provides gradient clipping (clip\_norm=5.0) and general-purpose Laplacian noise addition for market benchmark sanitization.

### Federated Averaging Function

The federated\_average() function computes a simple arithmetic mean of all client model state dictionaries. For each parameter key, all client values are summed and divided by the total number of clients, producing the new global model state.

! Important: The LLM (GGUF model) does NOT train the forecasting model. The LSTM is the sole forecasting engine. The LLM is used exclusively for strategic explanations and chat.

## 6. LSTM Forecasting Model

The demand forecasting model is a Long Short-Term Memory (LSTM) neural network implemented in PyTorch. It predicts the next week's demand using a sliding window of the previous 5 demand data points.

### Model Architecture

|            |                               |
|------------|-------------------------------|
| Model Type | LSTM (Long Short-Term Memory) |
|------------|-------------------------------|

|                 |                                        |
|-----------------|----------------------------------------|
| Framework       | PyTorch (torch.nn)                     |
| Input Size      | 1 (univariate demand)                  |
| Hidden Size     | 50 neurons                             |
| Output Size     | 1 (next demand prediction)             |
| Sequence Length | 5 time steps                           |
| Batch Format    | (batch, seq_len, input_size)           |
| Decoder         | Linear layer on last LSTM hidden state |
| Loss Function   | MSELoss                                |
| Optimizer       | Adam (lr=0.01)                         |
| Local Epochs    | 5 per federated round                  |

## Data Preprocessing

Demand values are normalized by dividing by the maximum demand across all participating clients. This shared normalization scale ensures consistent model training across the federation. Sequences are created using a sliding window approach:

```
# Sliding window sequence creation
for i in range(len(data_norm) - sequence_length):
    X.append(data_norm[i:i+sequence_length])
    y.append(data_norm[i+sequence_length])

# Reshape for LSTM: (Batch, Seq=5, Feature=1)
X = tensor(X).unsqueeze(-1)
y = tensor(y).unsqueeze(-1)
```



## 7. Optimization Engine

After generating demand forecasts, the optimization engine computes recommended order quantities incorporating multiple supply chain factors.

### Optimization Parameters

|                     |                                     |
|---------------------|-------------------------------------|
| Forecasted Demand   | From LSTM model output              |
| Current Inventory   | Default 50 units                    |
| Disruption Risk     | From dataset disruption_prob column |
| Emission Factor     | From dataset emission_factor column |
| Carbon Cap          | 500.0 (configurable)                |
| Selling Price       | \$4.00/unit                         |
| Cost Price          | \$1.50/unit                         |
| Waste/Spoilage Cost | \$0.50/unit                         |

### Order Quantity Formula

```
# Safety Stock with risk buffer
safety_stock = int(forecast * (0.1 + risk))

# Order Quantity (if no human override)
order_qty = max(0, forecast + safety_stock - inventory)

# Emissions calculation
emissions = order_qty * emission_factor
feasible = (emissions <= carbon_cap)
```

### Financial Projections

The engine computes comprehensive financial projections for each order decision:

```
# Financial calculations
available_stock = inventory + order_qty
projected_sales = min(forecast, available_stock)
unsold_stock = max(0, available_stock - projected_sales)

revenue = projected_sales * selling_price
order_cost = order_qty * cost_price
waste_cost = unsold_stock * waste_cost_per_unit
net_profit = revenue - order_cost - waste_cost
```

### Human-in-the-Loop Override

The system supports real-time human override of the AI-recommended order quantity. When an operator adjusts the quantity, all financial projections, emissions calculations, and AI insights are dynamically recalculated. All decisions (approvals and

overrides) are persistently logged to `sc50_logs/decision_log.json` for audit trail purposes.

## 8. Privacy & Security Model

Privacy preservation is a foundational design principle. The system implements multiple layers of privacy protection to ensure that individual client data remains confidential.

### What Is Private

- A client cannot see another client's raw data
- Clients do not receive peer-by-peer rankings
- Clients receive only: their own metrics + aggregated market benchmarks + AI insight

### Privacy Mechanisms

| Mechanism            | Where Applied               | Purpose                            |
|----------------------|-----------------------------|------------------------------------|
| Laplacian Noise      | Weight updates (FedAvg)     | DP-protected model training        |
| DP Sanitization      | Market benchmark values     | Protect aggregate statistics       |
| Minimum Cohort       | Market benchmark generation | Prevent de-identification          |
| Fernet Encryption    | All network payloads        | Data confidentiality in transit    |
| HMAC-SHA256          | All secure envelopes        | Message integrity & authentication |
| Timestamp Validation | Network requests            | Prevent replay attacks             |
| Aggregation Only     | Market snapshots            | No raw peer data exposed           |

### Minimum Cohort Enforcement

The system requires a minimum of 3 clients (MIN\_PRIVACY\_COHORT) before any market benchmark data is generated or shared. This prevents scenarios where aggregate statistics could be reverse-engineered to identify individual participant data.

### DP Epsilon Configuration

The epsilon parameter controls the privacy-utility tradeoff:

- Lower epsilon (e.g., 0.1): Stronger privacy, more noise, less accurate benchmarks
- Higher epsilon (e.g., 20.0): Weaker privacy, less noise, more accurate benchmarks
- Default: 5.0 - Balanced privacy-utility tradeoff for demo purposes
- Configurable in real-time via the Control Tower sidebar slider

## 9. Secure Communication Protocol

All communication between the Control Tower and Client Nodes is secured through the `privacy_network.py` module, which implements a custom encrypted protocol.

### Encryption & Signing

- Symmetric key derived from SC\_SHARED\_SECRET using SHA-256 + base64 encoding
- Payloads encrypted with Fernet (AES-128-CBC with HMAC-SHA256 authentication)
- Secure envelopes signed with HMAC-SHA256 for integrity verification

- Timestamps embedded to enable stale payload rejection (max age: 900 seconds)

Network Endpoints

| Endpoint       | Method | Host  | Purpose                            |
|----------------|--------|-------|------------------------------------|
| Client /health | GET    | Node  | Health check with signed headers   |
| Client /sync   | POST   | Node  | Receive encrypted benchmark bundle |
| Tower /health  | GET    | Tower | Tower health verification          |
| Tower /upload  | POST   | Tower | Receive encrypted raw file upload  |

! Critical: Do NOT confuse the Streamlit UI port (e.g., 8503) with the secure node endpoint port (e.g., 8800). The Control Tower communicates with the secure endpoint port, not the browser UI port.

## 10. Dashboard & UI Design

Both control tower and client dashboards are built with Streamlit and feature a cohesive custom dark theme inspired by modern operations dashboards.

### Design System

| Token     | Value                | Usage                       |
|-----------|----------------------|-----------------------------|
| --bg-0    | #051F20              | Primary background          |
| --bg-1    | #0B2B26              | Sidebar / card background   |
| --card    | #0B2B26              | Panel backgrounds           |
| --line    | #163832              | Borders and dividers        |
| --text    | #DAF1DE              | Primary text color          |
| --muted   | #8EB69B              | Secondary/caption text      |
| --primary | #235347              | Buttons and active elements |
| --accent  | #8EB69B              | Highlights and labels       |
| --success | #1ed760              | Online/success indicators   |
| --danger  | #ff5252              | Offline/error indicators    |
| Font      | Inter (Google Fonts) | All UI text                 |

### UI Components

- Hero Banner: Project title, device info, model status, online node count
- Panel Cards: Bordered containers with shadow and rise-in animation
- Node Cards: SVG computer icon, status badge, latency, sync timestamps
- Metric Cards: Styled st.metric with dark backgrounds and muted labels
- Tab Navigation: Custom-styled tabs with active/inactive states
- Chat Interface: Styled chat messages with avatar color coding
- Progress Bar: Themed progress indicators for simulation pipeline
- Pill Badges: Compact status indicators on the hero banner

## 11. Control Tower Application

The Control Tower (app.py, 930 lines) is the central coordinator of the federated learning network. It manages model loading, simulation execution, node visibility, and human override decisions.

### Tab Structure

| Tab                | Features                                                     |
|--------------------|--------------------------------------------------------------|
| Control Center     | Model loading, simulation runner, data preview, metrics      |
| Node Network       | Node status, sync controls, upload endpoint, registry editor |
| Results & Override | Forecasts, financials, emissions, human approval controls    |
| AI Assistant       | Context-aware chat, strategic explanations, LLM interaction  |

## Simulation Pipeline

---

The `run_simulation_pipeline()` function orchestrates the complete federated learning workflow with progress indicators:

- Step 1 (0-15%): Load client data from DATASETS/ directory
- Step 2 (15-75%): Execute `FedSim.run()` for federated LSTM training
- Step 3 (75-100%): Build privacy-safe market bundles via `build_client_bundles()`
- Final: Extract primary client metrics and optimization results for display

## 12. Client Node Application

The Client Node App (client\_app.py, 432 lines) represents a single participant in the federated network. It provides private operations visibility, market comparison, raw data upload, and AI-powered insights.

### Tab Structure

| Tab         | Features                                                           |
|-------------|--------------------------------------------------------------------|
| Operations  | Node status, last seen, last sync, demand/supply/inventory metrics |
| Upload Data | Secure raw file upload (CSV/XLS/XLSX) to control tower             |
| Market View | Forecast/profit/emissions/risk vs market comparison                |
| AI Insight  | Private AI advisor using only client + market aggregate data       |

### Secure Node Server

Each client app automatically starts a secure HTTP endpoint (ensure\_node\_server) on the configured port. This endpoint handles:

- GET /health - Signed health checks from the Control Tower
- POST /sync - Receiving encrypted benchmark bundles from the tower
- Automatic registration in the node registry for the Control Tower to discover

### Market Comparison View

The Market View tab displays side-by-side metrics of the client's own performance versus privacy-safe market aggregates. Metrics include forecast, net profit, emissions, and risk level, each with a delta indicator showing the client's position relative to the market average. A combined demand chart shows the client's history overlaid with the market average demand trend.

## 13. Dataset Specifications

### Required Columns

| Column          | Type    | Description                   | Range      |
|-----------------|---------|-------------------------------|------------|
| demand          | integer | Weekly demand quantity        | >= 0       |
| disruption_prob | float   | Supply disruption probability | 0.0 to 1.0 |
| emission_factor | float   | Carbon emission per unit      | >= 0.01    |

### Optional Columns

| Column | Type    | Description               |
|--------|---------|---------------------------|
| week   | integer | Week number or time index |
| date   | string  | Date of the record        |
| brand  | string  | Brand/company name        |
| region | string  | Geographic region         |
| supply | integer | Supply quantity           |

|                 |         |                          |
|-----------------|---------|--------------------------|
| inventory_level | integer | Current inventory        |
| profit_margin   | float   | Profit margin percentage |

### Current Datasets

| File                            | Client                        | Size    |
|---------------------------------|-------------------------------|---------|
| client_0__upload__current.csv   | Client 0 (Uploaded)           | 61.8 KB |
| client_1_amul_gujarat.csv       | Client 1 - Amul Gujarat       | 61.8 KB |
| client_2_mother_dairy_delhi.csv | Client 2 - Mother Dairy Delhi | 63.3 KB |
| client_3_sudha_bihar.csv        | Client 3 - Sudha Bihar        | 60.2 KB |

### Synthetic Data Fallback

If insufficient real datasets are available, the system generates synthetic client data using a deterministic random generator (seed=42+cid) with trend, seasonality, and noise components over 52 weeks. This ensures the demo is always functional.



## 14. Local AI / LLM Integration

The system integrates a local GGUF language model for strategic AI insights, completely independent from the LSTM forecasting model. All inference runs locally on CPU.

### Model Configuration

|                |                                                          |
|----------------|----------------------------------------------------------|
| Model File     | models/SupplyChain-Qwen.gguf                             |
| Runtime        | llama.cpp CPU (llama-cpp-python or llama-completion.exe) |
| Context Window | 2048 tokens                                              |
| Thread Count   | CPU cores - 1                                            |
| GPU Layers     | 0 (forced CPU mode)                                      |
| Chat Format    | ChatML                                                   |
| Memory Mapping | Enabled (use_mmap=True)                                  |
| Temperature    | 0.7 (default)                                            |
| Top-p          | 0.9                                                      |
| Repeat Penalty | 1.1                                                      |

### Loading Strategy

The model loading follows a priority-based fallback chain:

- Priority 1: llama-cpp-python library (if installed) for native Python integration
- Priority 2: Bundled llama-completion.exe via subprocess (LlamaCppCLIModel wrapper)
- Priority 3: Bundled llama-cli.exe as secondary binary fallback
- Model path resolution scans multiple candidates including env vars and project paths

### Think Tag Stripping

The system includes a `_strip_think_tags()` function that removes `<think>...</think>` reasoning blocks from Qwen3 model output, ensuring clean user-facing responses.

### AI Usage Points

- Control Tower: Strategic explanation of order decisions in Results tab
- Control Tower: Chat-based Supply Chain Assistant in AI Assistant tab
- Client Node: Private AI insight generation from client + market aggregate data
- Main CLI: Command-line AI recommendation for forecast explanation

## 15. Environment & Configuration

### Environment Variables

| Variable | Purpose | Default |
|----------|---------|---------|
|----------|---------|---------|

|                       |                           |                                    |
|-----------------------|---------------------------|------------------------------------|
| SC_SHARED_SECRET      | Encryption/signing key    | Demo secret (change!)              |
| SC_CLIENT_ID          | Client identity index     | 0                                  |
| SC_NODE_PORT          | Secure node endpoint port | 8800 + client_id                   |
| SC_NODE_HOST          | Node bind host            | 127.0.0.1                          |
| SC_TOWER_HOST         | Tower endpoint host       | 127.0.0.1                          |
| SC_TOWER_PORT         | Tower endpoint port       | 9900                               |
| SC_LLM_GGUF_PATH      | Custom GGUF model path    | models/SupplyChain-Qwen.gguf       |
| SC_LLAMA_CPP_CLI_PATH | Custom llama CLI path     | Llama-cpp-cpu/llama-completion.exe |

### Generated Runtime Files

| File                        | Location                | Purpose                            |
|-----------------------------|-------------------------|------------------------------------|
| decision_log.json           | sc50_logs/              | Audit trail of approvals/overrides |
| node_registry.json          | sc50_logs/              | Known node endpoints and states    |
| control_tower_endpoint.json | sc50_logs/              | Tower upload endpoint info         |
| latest_bundle.secure.json   | sc50_logs/client_inbox/ | Encrypted client bundle            |
| client_N__upload__current.* | DATASETS/               | Latest uploaded raw datasets       |

## 16. Installation & Deployment

### Prerequisites

- Python 3.10 or higher recommended
- pip package manager
- llama.cpp CPU runtime (bundled) or llama-cpp-python package
- GGUF model file in models/ directory

### Dependency Installation

```
pip install -r requirements.txt
```

### Required Python Packages

| Package      | Purpose                                       |
|--------------|-----------------------------------------------|
| torch        | PyTorch for LSTM model and federated learning |
| transformers | Hugging Face utilities                        |
| accelerate   | Model loading acceleration                    |
| bitsandbytes | Quantization support                          |
| pandas       | Data manipulation and analysis                |
| numpy        | Numerical computation                         |
| streamlit    | Web dashboard framework                       |
| matplotlib   | Financial visualization (pie charts)          |
| cryptography | Fernet encryption for secure communication    |
| openpyxl     | Excel .xlsx file reading                      |
| xlrd         | Legacy .xls file reading                      |

### Running the System

Start the Control Tower:

```
python -m streamlit run app.py
```

Start a Client Node (PowerShell):

```
$env:SC_CLIENT_ID="0"  
$env:SC_NODE_PORT="8800"  
python -m streamlit run client_app.py --server.port 8503
```

Start additional Client Nodes:

```
# Client 1:
$env:SC_CLIENT_ID="1"
$env:SC_NODE_PORT="8801"
python -m streamlit run client_app.py --server.port 8504

# Client 2:
$env:SC_CLIENT_ID="2"
$env:SC_NODE_PORT="8802"
python -m streamlit run client_app.py --server.port 8505
```

## 17. Recommended Demo Workflow

Follow this step-by-step workflow for a complete demonstration of the system.

### Step 1: Start Control Tower

Run 'python -m streamlit run app.py'. The tower UI opens on <http://localhost:8501> and automatically starts a secure endpoint on port 9900.

### Step 2: Start Client Nodes

Set SC\_CLIENT\_ID and SC\_NODE\_PORT environment variables, then run the client app on separate Streamlit ports (e.g., 8503, 8504).

### Step 3: Upload Client Data

In each client app, go to 'Upload Data' tab, select a raw CSV/XLS/XLSX file, and click 'Send Raw File to Control Tower'.

### Step 4: Verify Node Status

In the Control Tower, go to 'Node Network' tab and click 'Refresh Node Status'. Verify all nodes show as Online.

### Step 5: Load AI Model

In the Control Tower, go to 'Control Center' and click 'Load Local Model' to initialize the GGUF language model.

### Step 6: Run Federated Simulation

Click 'Run Federated Simulation' to execute the federated LSTM training across all client datasets. Monitor progress and metrics.

### Step 7: Sync Client Bundles

Click 'Secure Sync Online Nodes' to push encrypted benchmark bundles to all online client nodes.

### Step 8: Review Market Comparison

In each client app, open 'Market View' to see private metrics versus privacy-safe market aggregates.

### Step 9: Generate AI Insights

In the client app, load the local model and click 'Generate Client AI Insight' for personalized strategic advice.

### Step 10: Approve or Override

In the Control Tower 'Results and Override' tab, review the forecast and adjust order quantity if needed. Click 'Approve Decision' to log.

## 18. Troubleshooting Guide

### Node shows Online in browser but Tower says Offline

The tower is trying the Streamlit UI port instead of the secure node port. Ensure the node registry points to the SC\_NODE\_PORT (e.g., 8800), not the browser UI port (e.g., 8503).

### Uploaded file is ignored during simulation

Verify: (1) File extension is .csv, .xls, or .xlsx (2) Required columns exist (demand, disruption\_prob, emission\_factor) (3) Upload succeeded (4) New simulation was run after uploading.

### GGUF model fails to load

Check: (1) models/SupplyChain-Qwen.gguf file exists (2) Llama-cpp-cpu/ directory exists with llama-completion.exe (3) All Python dependencies are installed.

### Excel file fails to load

Ensure openpyxl (for .xlsx) and xlrd (for .xls) are installed. Verify the file is not corrupted and column names are correct.

### Market comparison is unavailable

The system requires at least 3 clients (MIN\_PRIVACY\_COHORT) before exposing market benchmarks. Add more client datasets or reduce the cohort requirement.

### Secure sync fails between tower and nodes

Verify both tower and node use the same SC\_SHARED\_SECRET. Check that the tower registry URL matches the actual node endpoint. Ensure no firewall blocking.

## 19. Technical Specifications

### Codebase Metrics

|                                  |                                         |
|----------------------------------|-----------------------------------------|
| Total Python Source Lines        | ~3,000 lines across 5 modules           |
| app.py (Control Tower)           | 930 lines                               |
| main.py (Core Logic)             | 723 lines                               |
| privacy_network.py (Security)    | 555 lines                               |
| client_app.py (Client Node)      | 432 lines                               |
| dashboard_data.py (Benchmarking) | 361 lines                               |
| Total Dataset Files              | 4 files (~247 KB total)                 |
| Supported File Types             | CSV, XLS, XLSX                          |
| Server Threading                 | ThreadingHTTPServer with daemon threads |

|                     |                                    |
|---------------------|------------------------------------|
| Encryption Standard | Fernet (AES-128-CBC + HMAC-SHA256) |
|---------------------|------------------------------------|

Performance Characteristics

|                      |                                         |
|----------------------|-----------------------------------------|
| LSTM Training        | ~2-5 seconds per round (CPU, 3 clients) |
| Market Bundle Build  | < 1 second                              |
| Health Check Latency | < 100ms (local network)                 |
| Node Server Startup  | < 500ms per node                        |
| LLM Inference        | 5-30 seconds per response (CPU, GGUF)   |
| Max Payload Age      | 900 seconds (secure envelope)           |
| Simulation Progress  | Streaming progress bar (0-100%)         |

## 20. Future Roadmap

While the current implementation provides a strong foundation, several extensions can enhance the system for production-grade deployment:

### Infrastructure Enhancements

- Real multi-machine deployment with proper network isolation
- Database-backed storage (replacing JSON file logs)
- Container-based deployment (Docker/Kubernetes)
- Real-time node telemetry and monitoring dashboards
- Load balancing for multiple simultaneous simulations

### Security Improvements

- TLS/HTTPS for all tower-node communication
- Mutual TLS (mTLS) authentication between nodes
- Role-based access control (RBAC) for tower operations
- Key rotation and management infrastructure
- Audit logging with tamper-evident records

### Machine Learning Advances

- Advanced aggregation algorithms (FedProx, FedMA, SCAFFOLD)
- Asynchronous federated learning for heterogeneous nodes
- Multi-product and multi-category forecasting models
- Automated hyperparameter tuning for LSTM architecture
- Model versioning and A/B testing framework

### Supply Chain Extensions

- Multi-tier supply chain modeling (farm to retailer)
- Real-time IoT sensor data integration for cold chain
- Route optimization and logistics planning
- Supplier risk scoring with external data sources
- Regulatory compliance reporting (food safety standards)

> This project demonstrates the practical application of federated learning in supply chain management, combining privacy-preserving machine learning, secure communication protocols, and AI-powered decision support into a cohesive, production-inspired system.